

Working with Codex

In crew 1.0, the interactive Claude session implements and Codex reviews. Both agents read the same instructions, and the same context hook can run under both. This guide explains how to set up the Codex half and what is proven about it so far.

What Each Side Reads

Surface	Claude Code	Codex
AGENTS.md (80 lines or fewer, generated)	through @AGENTS.md in CLAUDE.md	natively
.claude/rules/<subsystem>.md (paths:-scoped)	yes, when a matching file is read	no (Codex has no path-scoped rules)
crew's context hook	plugins hooks.json	.codex/hooks.json (generated)
CLAUDE.md	natively	fallback via project_doc_fallback_filenames

Codex has no path-scoped rules, so its hook's PostToolUse slice is the only per-file channel.

Generate the Files

Run these from the repository root. Each command writes nothing if the files are already current. With --check, it writes nothing at all and exits 1 on drift.

```
S="<crew plugin root>/hooks/scripts"

python3 "$S/crew_instructions.py" agents      # AGENTS.md; only the crew:keep block
is yours

python3 "$S/crew_instructions.py" rules      # .claude/rules/<subsystem>.md
from .crew/codemap/

python3 "$S/crew_instructions.py" codex      # .codex/hooks.json and
.codex/config.toml
```

```
python3 "$S/crew_instructions.py" codex --check
```

- `.codex/hooks.json` is built from the same hook table as crew's Claude registration (`H00K_TABLE` in `crew_instructions.py`). Each entry has a `command` (`bash`) and a `commandWindows` (`PowerShell`, `-Harness codex`). The paths inside are absolute and belong to one machine, so regenerate the file on each machine rather than committing it.
- `.codex/config.toml` sets `project_doc_fallback_filenames` = `["CLAUDE.md"]` and two profiles: `review` (`sandbox_mode` = `"read-only"`, `approval_policy` = `"never"`) and `work` (`workspace-write`). Codex reads project config only for a **trusted** project.

Review and Work

```
codex exec --profile review --json --sandbox read-only "<review prompt>" # the
reviewer

codex exec --profile work "<task>" # only if
you choose Codex to implement
```

A review that exits non-zero or prints nothing is INCOMPLETE, never CLEAN.

What Is Proven, and What Is Only Configured

Run the probe. The sample below shows the probe's format with this build's values. The hooks feature value was measured with `codex features list`. The probe itself has only been run against a stub codex in the test suite (`tests/test_crew_instructions.py`).

```
python3 "$S/crew_instructions.py" codex-probe

codex: /usr/local/bin/codex (codex-cli 0.155.1)

hooks feature: enabled

project files: current

PostToolUse: configured, not proven

SessionStart: configured, not proven

SubagentStart: configured, not proven
```

UserPromptSubmit: configured, not proven

- **Established by reading the codex 0.155.1 binary, not its docs:** the hook events (SessionStart, UserPromptSubmit, PostToolUse, SubagentStart and others), the handler keys command, commandWindows, timeout and additionalContextLimit, and a SubagentStartHookSpecificOutputWire output type. `codex features list` reports `hooks stable true`.
- **Not measured:** whether a hook's `additionalContext` reaches the Codex model, and whether Codex honours profiles from a project's `.codex/config.toml`. A live test run would need Codex to write its session state under `~/.codex`, and this build did not do that.
- The probe changes an event's line from "configured, not proven" to "hook invoked Nx under Codex" only when the context log records an invocation under the codex harness. Even then it claims only that the hook ran, not that the model received the context. To prove delivery, rerun the seeded-note test from [Confirming recall reaches your sessions](#) with `codex exec` in place of `claude -p`.

When Codex Is Barred

A review is independent only when it comes from a different model family than the author. If Codex wrote the diff, it cannot review that diff. The order falls through to Copilot, then to Claude's reviewer.